CHAPTER 4: SYSTEM DESIGN AND DEVELOPMENT METHODOLOGY

HavenSec Arena
AI-Powered Multi-Agent Red Team & Blue Team Cybersecurity Simulation Platform

Note: This chapter describes the proposed system design and development methodology. Features are described as proposed/planned unless demonstrated later in the implementation and testing chapters.

4.1 Introduction

HavenSec Arena is proposed as an AI-powered multi-agent cybersecurity simulation platform. The system is designed to provide a controlled environment in which Red Team activities, Blue Team defensive activities and objective evaluation can be coordinated within an isolated cyber-range. The design emphasizes safe experimentation by using simulated events, synthetic or dummy data and sandboxed resources rather than live production targets.

The architecture separates responsibilities among specialized agents. An Orchestrator manages the scenario lifecycle, Red Team agents perform controlled attack simulation, Blue Team agents support monitoring and response, and a Referee Agent evaluates the exercise. IBM Bob is proposed as the core AI reasoning component for agent-level reasoning and coordination. The output includes an incident timeline, evaluation metrics and a composite cyber-resilience score.

The design follows a modular approach so that additional scenarios, agents and scoring rules can be introduced without redesigning the complete platform.

4.2 Proposed System Architecture

The proposed architecture consists of an AI reasoning layer, specialized multi-agent roles, an isolated cyber-range and an evaluation/reporting layer. Simulated offensive actions remain confined to the sandbox, while defensive analysis is based on telemetry generated within that environment.

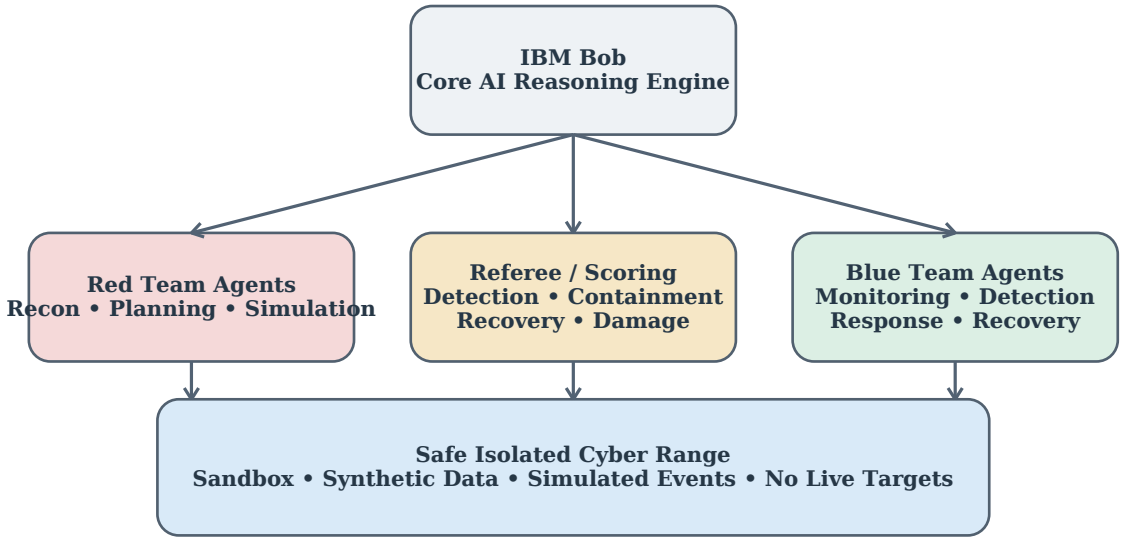


Figure 4.2: Proposed System Architecture

Main architectural components:

• IBM Bob – Core AI Reasoning Engine: proposed to provide reasoning support and coordination for the agent workflow.

• Orchestrator Agent: manages scenario initialization, agent communication and simulation state.

• Red Team Agents: perform controlled reconnaissance, attack-path planning and simulated execution.

• Blue Team Agents: monitor telemetry, detect suspicious events, recommend or perform predefined defensive actions, and support recovery.

• Referee / Scoring Agent: evaluates the exercise using detection time, containment, simulated damage and recovery.

• Safe Isolated Cyber Range: contains the simulation using sandboxed resources, synthetic data and simulated events.

• Timeline and Reporting Layer: converts events and decisions into an incident timeline, score and security report.

4.3 Block Diagram

For the proposed software system, the block diagram represents the movement of control, simulated activity and evaluation information between the major modules.

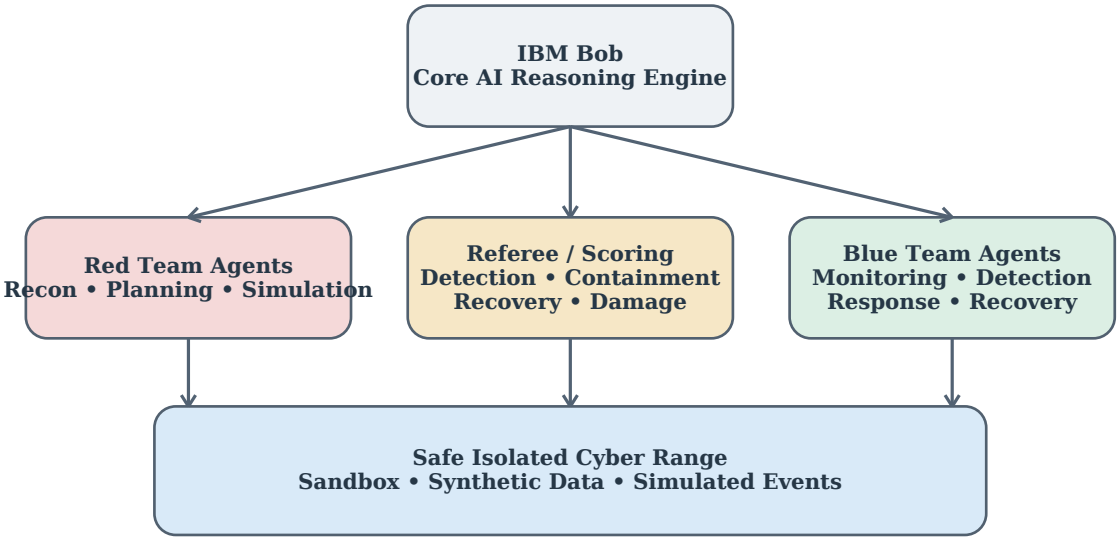


Figure 4.3: Proposed System Module / Block Diagram

The Scenario Manager initializes the exercise and passes the scenario to the Red Team Simulation module. Simulated events are produced inside the Sandbox. The Blue Team Defense module consumes telemetry and produces defensive decisions. The Referee evaluates the exercise. The Timeline and Report modules present the results.

4.4 Hardware Requirements

HavenSec Arena is primarily a software project. No dedicated electronic hardware is required. Development and execution can be performed on a capable personal computer or workstation with sufficient resources for the selected sandbox environment.

Sr. No.	Component	Specification	Quantity	Purpose
1	Development computer/laptop	64-bit processor; 16 GB RAM recommended	1	Development, testing and execution
2	Network connectivity	Required for development tools or AI services when applicable	1	Development/service connectivity

Table 4.4: Proposed Hardware Requirements

4.5 Software Requirements

The following technologies are proposed. Final versions should be updated to match the actual implementation.

Software / Technology	Version	Purpose
Python	3.x	Backend logic, agent orchestration and simulation services
FastAPI	Implementation version	Backend API layer
React	18.x or implementation version	Web dashboard and interface
Tailwind CSS	Implementation version	Interface styling
Docker	Current stable version used	Containerized sandbox components
IBM Bob	Project environment	AI reasoning and agent assistance
Synthetic / dummy data	Project-generated	Safe simulation inputs and telemetry

Table 4.5: Proposed Software Requirements

4.6 Technology Selection

Python: suitable for backend logic, agent orchestration, data processing and security-simulation components.

FastAPI: provides a lightweight API layer connecting the frontend with agent and simulation services.

React: supports a dynamic dashboard for scenario state, events, agent activity and results.

Tailwind CSS: supports consistent and responsive interface styling.

Docker: supports isolated and reproducible sandbox components.

IBM Bob: proposed as the core AI reasoning component for the multi-agent workflow.

Synthetic Data: selected so that demonstrations do not require real credentials, sensitive records or production infrastructure.

4.7 System Modules

The proposed system is divided into modules with specific responsibilities and controlled interfaces.

4.7.1 Scenario Management Module

Purpose: Define and initialize a cybersecurity simulation scenario.
Input: Scenario type, difficulty, objectives and sandbox configuration.
Processing: Loads parameters, prepares dummy resources and initializes simulation state.
Output: Active scenario and execution context.

4.7.2 Red Team Simulation Module

Purpose: Simulate attacker behaviour in a controlled environment.
Input: Scenario objective and sandbox state.
Processing: Selects permitted simulation actions and generates controlled events.
Output: Simulated attack events and telemetry.

4.7.3 Blue Team Defense Module

Purpose: Detect, investigate and respond to simulated security events.
Input: Sandbox telemetry, logs and alerts.
Processing: Analyzes events and selects predefined or recommended defensive actions.

4.7 System Modules — Continued

4.7.3 Blue Team Defense Module — Output: Detection results, response actions and recovery status.

4.7.4 Referee and Scoring Module

Purpose: Independently evaluate the simulation.
Input: Event sequence, Red Team actions, Blue Team actions and timestamps.
Processing: Applies proposed scoring rules to measurable results.
Output: Metrics, composite score and observations.

4.7.5 Incident Timeline Module

Purpose: Present the chronological sequence of simulation events.
Input: Timestamped events and agent actions.
Processing: Orders and groups events by time and type.
Output: Incident timeline.

4.7.6 Dashboard and Reporting Module

Purpose: Present scenario status, agent activity and evaluation results.
Input: Simulation state, timeline and scoring data.
Processing: Converts results into visual summaries and reports.
Output: Dashboard, score visualization and security report.

4.8 Flowchart

The following flow represents the proposed end-to-end operation of HavenSec Arena.

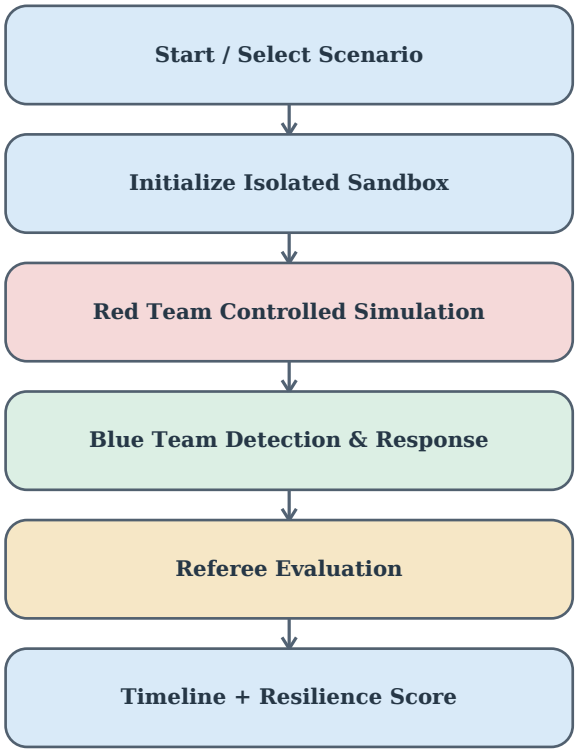


Figure 4.8: Proposed System Flowchart

The process begins when a scenario is selected. The Orchestrator prepares the sandbox and initializes the participating agents. The Red Team performs a controlled simulation, after which the Blue Team receives telemetry and performs detection and response. The Referee evaluates the complete sequence and the system produces an incident timeline and cyber-resilience score.

4.9 Algorithm

The following pseudocode describes the main simulation and evaluation workflow.

Step	Algorithm / Pseudocode
1	START
2	Select cybersecurity scenario and difficulty level.
3	Initialize isolated sandbox with synthetic/dummy data.
4	Initialize Orchestrator, Red Team, Blue Team and Referee agents.
5	Red Team selects a permitted simulation action based on scenario objectives.
6	Execute the controlled simulation action inside the sandbox.
7	Record generated event, telemetry and timestamp.
8	Send relevant telemetry to Blue Team agents.
9	Blue Team analyzes events and performs/recommends defensive action.
10	Record detection time, response action and containment result.
11	Referee evaluates Red Team and Blue Team actions.
12	Calculate proposed resilience metrics and composite score.
13	Generate incident timeline and security report.
14	Display results and improvement observations.
15	END

Table 4.9: Proposed HavenSec Arena Algorithm

4.10 UML / DFD / ER Diagram

Because HavenSec Arena is primarily a software system, UML and data-flow representations are applicable. The final graphical diagrams should be updated to match the actual implementation.

Diagram	Main Elements	Purpose
Use Case Diagram	User, Scenario Manager, Red Team, Blue Team, Referee, Dashboard	Shows interactions with the system
Activity Diagram	Scenario selection, simulation, detection, response, scoring	Shows sequence of activities
Sequence Diagram	Orchestrator, agents, sandbox, scoring service	Shows message flow
DFD	Scenario input, events, telemetry, evaluation, reports	Shows data movement

Table 4.10: Applicable UML / DFD Representations

4.11 Database Design

A database is optional for the initial prototype. If persistent storage is implemented, the following logical entities are proposed for storing scenario configuration, events and evaluation results.

Entity / Table	Important Fields	Purpose
Scenario	scenario_id, name, difficulty, objective, status	Stores scenario details
Agent	agent_id, role, type, status	Stores agent information
Event	event_id, scenario_id, timestamp, source, event_type, details	Stores simulation and telemetry events
Action	action_id, event_id, agent_id, action_type, timestamp, result	Stores agent actions
Evaluation	evaluation_id, scenario_id, detection_time, containment, recovery, damage, score	Stores Referee evaluation
Report	report_id, scenario_id, timeline, findings, recommendations	Stores generated results

Table 4.11: Proposed Database Design

4.12 Development Methodology

An iterative development methodology is suitable for HavenSec Arena because the project contains several interconnected modules and the agent workflow may require repeated refinement after testing. The system can be developed in small increments, with each iteration adding and validating a limited set of functions.

The proposed development cycle is:

• Requirement Analysis: identify simulation objectives, safety constraints and evaluation requirements.

• Architecture Design: define agents, sandbox boundaries, data flow and module interfaces.

• Prototype Development: implement the scenario manager, sandbox, agent workflow and reporting components.

• Integration: connect Red Team, Blue Team and Referee components.

• Testing: execute controlled scenarios and verify expected behavior.

• Refinement: modify workflow, scoring rules and interface based on observed results.

The iterative approach supports gradual expansion from a basic proof-of-concept to additional scenarios and more advanced agent behaviors.

4.13 Project Work Plan

Week	Planned Activity	Deliverable	Status
1	Requirement study and literature review	Requirement notes	Planned
2	Architecture and module design	Architecture diagrams	Planned
3	Sandbox and scenario prototype	Initial sandbox	Planned
4	Red Team and Blue Team workflow	Agent prototype	Planned
5	Referee scoring and timeline	Evaluation prototype	Planned
6	Dashboard and reporting	Integrated interface	Planned
7	Testing and refinement	Test results / modifications	Planned
8	Documentation and final integration	Final documentation	Planned

Table 4.13: Proposed Project Work Plan

4.14 Safety, Modularity and Design Considerations

The system design treats safety as a primary architectural constraint. Simulation activities are intended to remain inside the isolated cyber-range and use synthetic or dummy data. The design does not depend on real credentials, confidential organizational records, public targets or uncontrolled external infrastructure.

Modularity: Each major responsibility is separated into a module or agent. This makes it possible to change the scoring logic, add scenarios or replace a reasoning component without redesigning the entire system.

Traceability: Timestamped events and agent actions provide the basis for an incident timeline and objective evaluation.

Extensibility: Future versions can add adaptive difficulty, additional agents, richer scenarios and integrations with selected security tools after appropriate validation.

AI limitation: IBM Bob is treated as a proposed external AI technology for reasoning or orchestration support, not as an already-existing dedicated cyber-range engine. AI-generated decisions require validation.

4.15 Expected System Outputs

Output	Description
Scenario State	Selected scenario, difficulty, objectives and current execution state
Event / Telemetry Stream	Timestamped synthetic events generated during the simulation
Agent Activity	Recorded Red Team, Blue Team and Referee actions
Incident Timeline	Chronological representation of the simulated exercise
Evaluation Metrics	Detection time, containment, simulated damage and recovery
Cyber-Resilience Score	Composite score generated from the selected metrics
Security Report	Findings, observations and improvement recommendations

Table 4.14: Expected System Outputs

4.16 Chapter Summary

This chapter presented the proposed system design and development methodology for HavenSec Arena. The architecture separates Red Team simulation, Blue Team defense, Referee evaluation and orchestration responsibilities while keeping simulated activity within an isolated cyber-range.

The chapter defined the proposed hardware and software requirements, technology selection, system modules, workflow, algorithm, applicable UML/DFD representations, database entities, development methodology, safety considerations and project work plan.

The architecture is intentionally modular and emphasizes controlled simulation, synthetic data, event traceability and measurable evaluation. The next implementation and testing stages should convert these proposed components into working modules and validate them using controlled scenarios.

Implementation note: Actual implementation details, screenshots, measured testing results, observed issues and final modifications should be documented in the subsequent chapters. The proposed architecture should be updated wherever the implemented system differs from this design.

Chapter 4 Conclusion

HavenSec Arena provides a structured blueprint for an academic cyber-resilience simulation platform. Its proposed contribution is the integration of controlled Red Team simulation, Blue Team defensive response, objective Referee scoring, AI-assisted orchestration and improvement-oriented reporting within a safe sandbox.